DAY 1 - PYTHON FUNDAMENTALS

Python for AWS Data Engineering

Objective: Build a strong Python foundation before moving to CSV, JSON, APIs, Pandas, PySpark, AWS S3, Lambda, Glue and Boto3.

1. Python in Data Engineering

Python is a programming language widely used in Data Engineering to read, clean, validate, transform and process data, automate jobs, call APIs, interact with AWS services and build ETL pipelines.

Example input data:

```
customer_id,name,amount
101,Arun,5000
102,Priya,7000
103,Rahul,3000
```

Typical AWS Data Engineering flow:

```
Receive File
    |
Read Data
    |
Validate Data
    |
Remove Bad Records
    |
Transform Data
    |
Write Output
    |
Amazon S3
    |
AWS Glue / PySpark
    |
Amazon Redshift
```

2. First Python Program - print()

The print() function displays information as output.

```
print("Hello Data Engineering")
```

Output:

```
Hello Data Engineering
```

```
print("Welcome to AWS Data Engineering")
print("Learning Python")
print("Day 1")
```

Output:

```
Welcome to AWS Data Engineering
Learning Python
Day 1
```

Data Engineering example:

```
print("Pipeline Started")
print("Reading customer data")
print("Processing completed")
```

Output:

```
Pipeline Started
```

```
Reading customer data
Processing completed
```

3. Variables

A variable is a name used to store a value.

```
name = "Arun"
age = 30
salary = 50000

print(name)
print(age)
print(salary)
```

Data Engineering example:

```
source_file = "customers.csv"
bucket_name = "company-raw-data"
record_count = 1000

print(source_file)
print(bucket_name)
print(record_count)
```

Output:

```
customers.csv
company-raw-data
1000
```

Part	Example	Meaning
Variable Name	source_file	Name used to store the value
Assignment	=	Assigns a value
Value	"customers.csv"	Actual data stored

Variable Naming

Correct:

```
customer_name = "Arun"
customer_id = 101
total_amount = 5000
```

Incorrect:

```
1name = "Arun"
customer name = "Arun"
```

Python is case-sensitive. name and Name are treated as different variables.

```
name = "Arun"
Name = "Priya"

print(name)
print(Name)
```

4. Basic Data Types

```
customer_name = "Arun"
customer_id = 101
account_balance = 25000.50
is_active = True
```

Value	Python Type	Meaning	Data Example
"Arun"	str	String / text	Customer name

101	int	Whole number	Customer ID
25000.50	float	Decimal number	Amount
True	bool	True / False	Active status

Use type() to check the data type:

```
print(type(customer_name))
print(type(customer_id))
print(type(account_balance))
print(type(is_active))
```

Output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Dataset example:

customer_id	customer_name	amount	active
101	Arun	5000.50	True
102	Priya	7000.00	False

Python types: customer_id -> int | customer_name -> str | amount -> float | active -> bool

5. Type Conversion

A value can look like a number but still be stored as a string.

```
customer_id = "101"
print(type(customer_id))
```

Output:

```
<class 'str'>
```

Convert string to integer:

```
customer_id = int(customer_id)

print(customer_id)
print(type(customer_id))
```

Convert string to float:

```
amount = "5000.50"
amount = float(amount)

print(amount)
print(type(amount))
```

Common conversions:

```
int("100")
float("500.50")
str(100)
bool(1)
```

Why conversion matters:

```
quantity = "10"
price = "500"

print(quantity + price)
```

Output:

```
10500
```

```
quantity = "10"
price = "500"
```

```
total = int(quantity) * int(price)
print(total)
```

Output:

```
5000
```

6. Operators

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Power

```
a = 10
b = 5

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Output:

```
15
5
50
2.0
```

Data pipeline calculation:

```
total_records = 1000
failed_records = 50

successful_records = total_records - failed_records
success_percentage = (successful_records / total_records) * 100

print(successful_records)
print(success_percentage)
```

Output:

```
950
95.0
```

7. Strings

Strings are used for names, file names, paths, S3 bucket names, JSON values and SQL text.

```
first_name = "Arun"
last_name = "Kumar"

full_name = first_name + " " + last_name
print(full_name)
```

Output:

```
Arun Kumar
```

Useful string methods:

Method	Example	Purpose

upper()	"aws".upper()	Convert to uppercase
lower()	"AWS".lower()	Convert to lowercase
title()	"aws data".title()	Title case
strip()	" Arun ".strip()	Remove unwanted spaces
replace()	".csv".replace(".csv",".json")	Replace text
startswith()	file.startswith("customer")	Check starting text
endswith()	file.endswith(".csv")	Check ending text

```
name = "aws data engineering"

print(name.upper())
print(name.lower())
print(name.title())
```

Output:

```
AWS DATA ENGINEERING
aws data engineering
Aws Data Engineering
```

```
customer_name = "   Arun   "
print(customer_name.strip())
```

Output:

```
Arun
```

```
file_name = "customer_data.csv"
new_name = file_name.replace(".csv", ".json")

print(new_name)
```

Output:

```
customer_data.json
```

```
file_name = "customer_2026.csv"
print(file_name.endswith(".csv"))
```

Output:

```
True
```

8. User Input - input()

input() accepts a value entered while the program is running. By default, input() returns a string.

```
name = input("Enter your name: ")
print("Hello", name)
```

```
age = input("Enter age: ")
print(type(age))
```

Output:

```
<class 'str'>
```

Convert numeric input when a number is required:

```
record_count = int(input("Enter record count: "))

print(record_count)
print(type(record_count))
```

9. Hands-on Practice

Scenario:

Source File	customers.csv

Total Records	10000
Failed Records	250
Processing Time	45.5 seconds

Calculate successful records and display the pipeline details.

```
source_file = "customers.csv"
total_records = 10000
failed_records = 250
processing_time = 45.5

successful_records = total_records - failed_records

print("Source File:", source_file)
print("Total Records:", total_records)
print("Successful Records:", successful_records)
print("Failed Records:", failed_records)
print("Processing Time:", processing_time)
```

Output:

```
Source File: customers.csv
Total Records: 10000
Successful Records: 9750
Failed Records: 250
Processing Time: 45.5
```

Calculate the success percentage:

```
total_records = 10000
failed_records = 250

successful_records = total_records - failed_records
success_percentage = (successful_records / total_records) * 100

print("Successful Records:", successful_records)
print("Success Percentage:", success_percentage)
```

Output:

```
Successful Records: 9750
Success Percentage: 97.5
```

10. Day 1 Quick Reference

Concept	Python
Display output	print()
Variables	customer_id = 101
String	str
Integer	int
Decimal	float
True / False	bool
Check type	type()
Convert to integer	int()
Convert to decimal	float()
Convert to string	str()
Remove spaces	strip()
Replace text	replace()

Check file extension	endswith()
User input	input()

11. Final Practice

Use the following data:

File Name	orders.csv
Total Records	50000
Failed Records	1250
Processing Time	120.5 seconds
Environment	production

Write a Python program to display the data and calculate:

- Successful Records
- Failed Records
- Success Percentage
- Failure Percentage